CARTESIANA — Guía de estudio para la defensa

Oct 28, 2026 · @Wenceslao José Villegas

La idea en una frase

Un programa de escritorio le manda órdenes de texto por una conexión TCP/IP a la máquina simulada en Webots, que las ejecuta y contesta; antes de moverse, la máquina busca sus sensores para saber dónde está.

En la defensa tienen que poder explicar con sus palabras, sin leer:

1. Quién es el **servidor** y quién el **cliente**, y por qué.
2. **Qué viaja** por la conexión: los comandos y las respuestas, con ejemplos.
3. Por qué la máquina necesita el **ciclo de inicialización** y cómo funciona.
4. Qué hace cada **archivo** importante.

El profesor lo dijo así: si no entienden el protocolo y la comunicación, es un *"error descalificante"*.

Conceptos de red

Concepto	Qué es	En este proyecto
TCP/IP	La forma en que viajan los datos por una red, la misma base que usa internet. TCP garantiza que los bytes lleguen completos y en orden	Toda la comunicación entre el programa y la máquina
Socket	El "enchufe" de software de cada extremo de una conexión TCP. Se abre, se envían y reciben bytes, y se cierra	Python: `socket.create_connection` en el programa y `socket.socket` + `bind` / `listen` / `accept` en Webots
Servidor	El que abre un puerto y **espera** conexiones	El controlador de Webots, que hace de PC industrial
Cliente	El que **inicia** la conexión hacia el servidor	El programa externo

Concepto	Qué es	En este proyecto
IP	La dirección de una computadora en la red	`127.0.0.1` significa "esta misma PC" (localhost)
Puerto	Un número que identifica a qué programa va dirigida la conexión dentro de esa PC	`5000`
Protocolo	Las reglas de qué se dice y cómo: formato de los mensajes, comandos y respuestas	El protocolo CARTESIANA v2, redactado por nosotros

¿Por qué Webots es el servidor? Porque reemplaza a la PC industrial de la máquina, que es la que queda escuchando a la espera de órdenes. El programa es el que decide cuándo conectarse y qué pedir.

¿Por qué no es una API REST? REST usa HTTP: cada pedido abre y cierra una conexión, con URLs y verbos como `GET` o `POST`. Acá usamos un **socket TCP crudo**: la conexión se abre una vez y queda abierta, y por ella pasan líneas de texto en los dos sentidos. Es más simple y es lo que pide el enunciado: *"mensajes de texto claro, regulados por un protocolo"*.

¿Por qué no hay puerto serie? Al principio entendimos que la máquina tenía un Arduino por USB y armamos un puerto COM virtual. El profesor aclaró que tiene una PC industrial que ya habla TCP/IP, así que esa capa sobraba y se quitó.

Qué viaja por el protocolo

Cada mensaje es **una línea de texto** terminada en salto de línea (`\n`). La regla es **un comando, una respuesta**: el programa manda una línea y espera la respuesta antes de mandar la siguiente. Las distancias van en milímetros. Esta es una sesión real (`>>` lo que manda el programa, `<<` lo que contesta la máquina):

#	Mensaje	Qué significa
1	`<< READY CARTESIANA 2.0 NOT_HOMED`	Saludo al conectarse: "soy CARTESIANA, protocolo 2.0, todavía no sé dónde estoy"
2	`>> MOVE 100 100 -40`	Pide ir a X=100, Y=100, Z=−40 mm
3	`<< ERR NOT_HOMED`	Rechazado: sin inicializar no se mueve
4	`>> HOME`	Pide el ciclo de inicialización
5	`<< DONE HOME`	Encontró los tres sensores: está en (0, 0, 0)

#	Mensaje	Qué significa
6	`>> SWITCH?` / `<< SWITCH X1 Y1 Z1`	Los tres sensores están activados (1 = activado)
7	`>> MOVE 100 100 -160`	Baja sobre el zócalo A
8	`<< DONE MOVE 100.0 100.0 -159.6`	Llegó; informa la posición medida por los sensores. Los 0,4 mm de diferencia están dentro de la tolerancia
9	`>> GRIP CLOSE` / `<< DONE GRIP CLOSE HELD`	Cerró el grip y tiene la pieza (`EMPTY` si no había nada)
10	`>> MOVE 400 200 -160` / `<< DONE MOVE …`	La lleva al zócalo B
11	`>> GRIP OPEN` / `<< DONE GRIP OPEN ZOCALO_B`	La soltó; quedó en el zócalo B
12	`>> MOVE -1 0 0` / `<< ERR RANGE X -1.0 (0..497)`	Fuera del recorrido: lo rechaza y no se mueve

Tres familias de respuesta para recordar:

- **`DONE …`**: terminó lo que se pidió.
- **`ERR …`**: no se pudo; el motivo va después (`NOT_HOMED`, `RANGE`, `SYNTAX`, `UNKNOWN`, `TIMEOUT`, `ABORTED`, `BUSY`).
- **Consultas inmediatas** (`PING`, `POS?`, `STATUS?`, `SWITCH?`, `INFO`, `STOP`): se responden al instante, aunque la máquina se esté moviendo. `STOP` tiene que poder cortar un movimiento en curso.

El ciclo de inicialización (homing)

Encoder relativo vs. absoluto. Un encoder **absoluto** sabe siempre en qué posición está. Uno **relativo**, como el de la máquina, solo cuenta cuánto se movió desde que se encendió. Al prenderla, el contador arranca en 0 en cualquier lugar, así que la máquina puede decir "estoy en 0" estando en medio del recorrido.

Sensor de inicio de carrera. Es un interruptor en un extremo de cada eje que se activa cuando el carro llega ahí. Es un punto fijo y conocido de la máquina.

El ciclo, eje por eje (X, después Y, después Z):

1. Moverse **rápido** (25 mm/s) hacia el sensor hasta que se active.
2. **Alejarse** 6 mm.

3. Volver **lento** (10 mm/s) y, en el instante en que el sensor se activa, decir "acá es el cero".

¿Por qué dos pasadas? La rápida encuentra el sensor sin perder tiempo, pero llega con inercia y se pasa un poco. La lenta se acerca suave, así el punto donde se activa el sensor es más preciso y se repite siempre igual.

Cómo lo simulamos:

- Al dar Run, el controlador ubica los ejes en una **posición al azar**. Así nadie "sabe" dónde está.
- Hasta hacer `HOME`, responde `NOT_HOMED` y rechaza los `MOVE`.
- Los sensores son los **bloques amarillos** del modelo. Se simulan por posición: se activan cuando el carro llega a 3 mm del tope mecánico.
- Después del homing, X va de 0 a 497 mm, Y de 0 a 397 mm y Z de −177 a 0 mm. El cero de Z está arriba, así que bajar es negativo.
- Si se manda `STOP` en medio del homing, la máquina vuelve a quedar sin inicializar.

Cómo está hecho el código

Todo está en Python. Conviene abrir cada archivo y ubicar estas partes.

Lado máquina: `controllers/cartesiana_tcp/`

Parte	Qué hace
`protocol.py` → `parse()`	Convierte una línea recibida (`"MOVE 100 100 -40"`) en un comando con sus números, o devuelve el error (`ERR SYNTAX` , `ERR UNKNOWN`)
`protocol.py` → `LineBuffer`	TCP entrega bytes, no líneas: junta lo que llega y corta en cada `\n`
`cartesiana_tcp.py` → `_listen()`	Crea el socket servidor: `bind` al puerto 5000, `listen` y modo **no bloqueante**
`_poll_network()`	En cada paso de simulación (16 ms): acepta un cliente si hay, lee lo que llegó y procesa las líneas completas. No bloqueante = Webots nunca se congela esperando datos
`_run_immediate()`	Responde al instante `PING` , `POS?` , `SWITCH?` , `STOP` …

Parte	Qué hace
Cola (`self.queue`) + `_start()` / `_update()`	Los comandos de movimiento se ejecutan de a uno. `_start` lo arranca y `_update` revisa en cada paso si ya terminó; al terminar, envía `DONE`
`_update_home()`	Máquina de estados del homing: fases `fast` → `backoff` → `slow`, eje por eje
`_power_on()`	Al arrancar, pone los ejes en una posición al azar (encoder relativo)
`serve()`	El bucle principal: paso de simulación → red → cola → pieza → etiquetas

Lado programa: `programa_externo/`

Parte	Qué hace
`enlace.py` → `abrir()`	Abre el socket cliente hacia IP:puerto
`enlace.py` → `_leer()`	Hilo que lee del socket todo el tiempo y arma las líneas
`enlace.py` → `comando()`	Envía una línea y **espera su respuesta**: es la regla "un comando, una respuesta"
`cartesiana_app.py` → `_worker()`	Hilo que habla con la máquina, para que la ventana no se congele mientras espera
`_inicializar()`	Botón Inicializar: envía `HOME`
`_guardar_punto()` / `_ejecutar()`	Graba la posición actual y reproduce la lista: `MOVE`, después `GRIP` y `WAIT` si corresponde

Por qué hay hilos (threads): si la ventana esperara la respuesta de un `MOVE` que tarda 10 segundos, se colgaría. Por eso un hilo aparte espera la respuesta y la ventana sigue respondiendo, por ejemplo para apretar STOP.

Preguntas probables del profesor

Practiquen en voz alta: tapen la respuesta y contesten con sus palabras.

1. ¿Qué viaja por la conexión? Líneas de texto: el programa manda comandos como `MOVE 100 100 -40`, `GRIP CLOSE` o `HOME`, y la máquina contesta `DONE …`, `ERR …` o datos como `POS …`. Una línea por mensaje, en milímetros.

2. ¿Quién es el servidor y quién el cliente? El servidor es Webots, que hace de PC industrial y escucha en el puerto 5000. El cliente es el programa externo, que se conecta a `127.0.0.1:5000`.

3. ¿Por qué TCP y no UDP? TCP garantiza que los mensajes lleguen completos, sin pérdidas y en orden. Para órdenes de movimiento no se puede perder ni desordenar un comando.

4. ¿Qué pasa si mando un MOVE antes de inicializar? Responde `ERR NOT_HOMED` y no se mueve. Sin los sensores no sabe dónde está, así que moverse sería peligroso.

5. ¿Por qué hace falta el homing si Webots ya sabe la posición? Porque la máquina real tiene encoders relativos y no lo sabe. El gemelo digital tiene que comportarse como la real: arrancamos en una posición al azar y obligamos a inicializar.

6. ¿Cómo sabe el programa que un movimiento terminó? Espera la respuesta `DONE MOVE`. La máquina la manda cuando los tres ejes quedan a menos de 0,8 mm del objetivo durante 0,2 s.

7. ¿Qué pasa si pido una posición fuera del recorrido? `ERR RANGE` con el eje y el rango permitido, y no se mueve: la orden se rechaza en vez de recortarse.

8. ¿Cómo funciona STOP si la máquina está ocupada? STOP es un comando inmediato: se atiende aunque haya un movimiento en curso. Frena donde está, responde `ERR ABORTED` por lo que cortó y después `OK STOP`.

9. ¿Por qué la simulación no se congela esperando mensajes? Porque el socket es no bloqueante: en cada paso de 16 ms se fija si llegó algo, y si no, sigue.

10. ¿Cómo se graba y reproduce una trayectoria? Se mueve la máquina a mano, se aprieta "Guardar punto actual" (posición + acción de grip + espera) y después "Ejecutar" manda un `MOVE` por punto, esperando cada `DONE`. Se guarda en un archivo JSON.

11. ¿Cómo agarra la pieza? Los dos dedos tienen motor propio. Si al cerrar la pieza está entre los dedos, el controlador (que es Supervisor) la mueve junto con la punta. Lo hicimos así porque la fricción simulada no es confiable para piezas chicas.

12. ¿Qué pasa si Webots corre en otra computadora? En el programa se pone la IP de esa PC en vez de `127.0.0.1`, y en esa PC se abre el puerto 5000 en el firewall.

13. ¿Qué hizo la IA y qué entienden ustedes? Respondan con honestidad: la IA ayudó a escribir el código. Ustedes definieron qué debía hacer, lo probaron y lo corrigieron con la devolución del profesor (sacar el puerto serie, agregar el homing). Esta guía es para poder

explicar cada parte.

Guion de la demo (12 minutos)

Minutos	Qué mostrar	Qué decir
0–2	El modelo en Webots	Qué es CARTESIANA, los tres ejes, el grip, los sensores amarillos y los zócalos de ejemplo
2–4	Diagrama de conexión	Cliente y servidor, TCP/IP, puerto 5000, por qué no REST ni puerto serie
4–5	Webots en **Run** + programa **Conectar**	En el registro se ve `READY …` `NOT_HOMED` y `PONG`
5–6	Intentar mover sin inicializar	Aparece `ERR NOT_HOMED`: explicar el encoder relativo
6–7	**Inicializar**	Mirar a X, Y y Z buscar sus sensores; el indicador pasa a verde
7–10	Grabar a mano: ir al cubo, cerrar, subir, ir al azul, bajar, abrir, guardando cada punto. Después, **Ejecutar**	Leer en voz alta algunos mensajes del registro: `MOVE`, `DONE`, `GRIP CLOSE HELD`
10–11	**STOP** a mitad de una ejecución	`ERR ABORTED` + `OK STOP`
11–12	Cierre	Qué se supuso (medidas, sensores, grip) y qué se mediría en la máquina real

Antes de la defensa:

- [ ] Correr la demo completa al menos dos veces en la PC que van a usar
- [ ] Tener abierto el registro de mensajes del programa, bien visible
- [ ] Tener a mano `docs/protocolo_cartesiana.md` por si piden un comando que no recuerden
- [ ] Repartir entre los integrantes quién explica cada parte, y que todos puedan responder las 13 preguntas